Business: Online Fashion Store (StyleSphere)

StyleSphere is a fictional e-commerce fashion retailer selling men's and women's apparel, footwear, and accessories. Customers browse products, place orders, leave reviews, and receive shipments through an online delivery system.

Target Customers:

Young adults aged 18–35 who shop online for affordable, trend-driven clothing and prefer fast delivery and personalized recommendations.

Revenue Streams:

- Direct product sales (primary revenue)
- Premium subscription program (free shipping + exclusive perks)
  Sponsored product placements on the homepage
- Occasional return-processing fees

How Data Supports Operations:

StyleSphere uses data to:

- Forecast demand and restock best-selling items
- Segment customers for targeted marketing
- Analyze site behavior (clicks, conversions, abandoned carts)
- Manage inventory to reduce stockouts and overstock
- Monitor return rates to identify defective/low-performing products

These analytics help guide budgeting, buying decisions, product assortment, and marketing.

Data Required for Daily Operations & Record-Keeping:

- Customer profiles and contact information
- Product catalog details
- Inventory levels by warehouse
- Order, payment, and shipment history
- Return and refund records

Data Used for Reporting:

- Monthly revenue and order volume
- Top-selling categories/products
- Return rates and reasons
- Customer lifetime value
- Supplier performance and restock times

Compliance-Required Data:

- Customer contact information (receipts, notifications)
- Payment transaction details (audits)
- Return/refund documentation
- Product safety information

Front-End Data Customers Need to See:

- Product details (price, size, color, availability)
- Order history
- Delivery status and tracking
- Product reviews and ratings

Internal Data for Decision-Making:

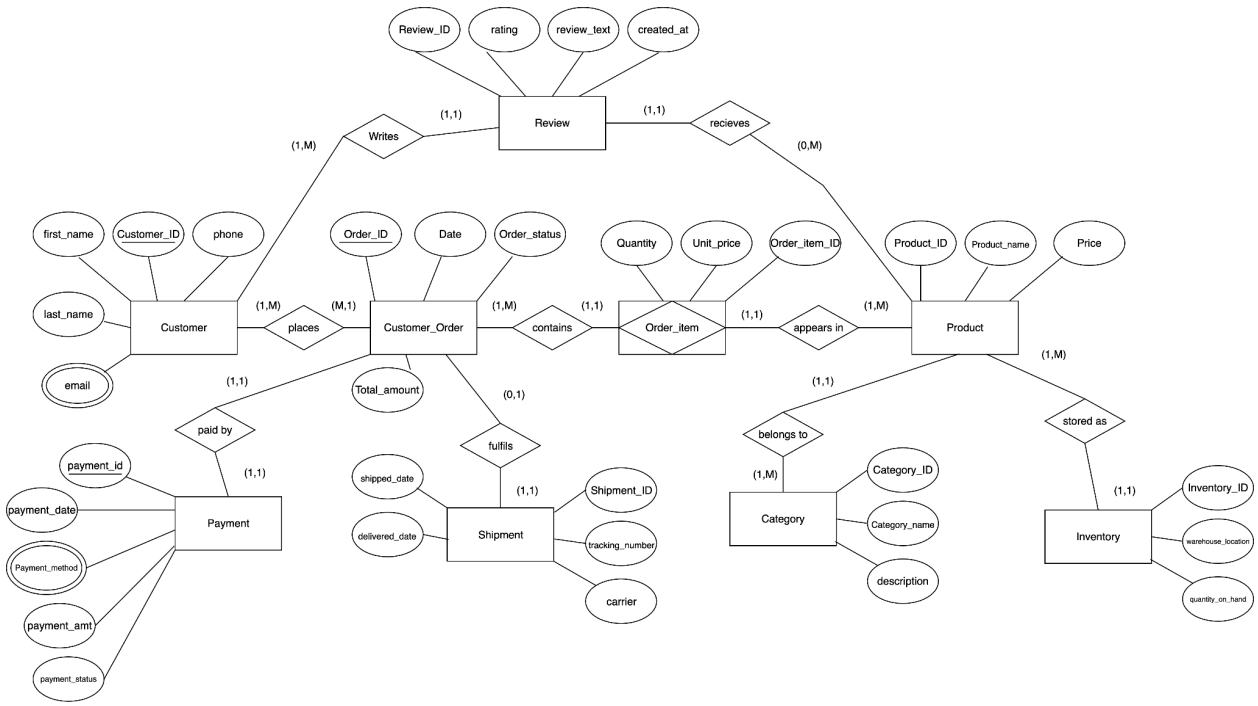
- Inventory performance
- Customer behavior trends
- Marketing attribution
- Supplier and warehouse performance
- Profit margins

Data the business will keep track of

For the StyleSphere online fashion store, the database will store:

- Customers – who is buying
- Products – what is being sold
- Categories – how products are grouped (tops, shoes, etc.)
- Customer_Orders – each purchase placed by a customer
- Order_Items – line items inside each order
- Payments – how each order was paid
- Shipments – shipping and tracking info
- Inventory – stock levels by product
- Reviews – product ratings and feedback from customers

ER Diagram



Relational Schema

Note:
PK = Primary Key
FK = Foreign Key

Customer(<u>customer_id(PK)</u>, first_name, last_name, email, phone)

Category(<u>category_id(PK)</u>, category_name, description)

Product(<u>product_id(PK)</u>, category_id (FK), product_name, price)

Customer_Order(<u>order_id(PK)</u>, customer_id(FK), order_date, order_status, total_amount)

Order_item(<u>order_item_id(PK)</u>, <u>order_id(FK)</u>, <u>product_id(FK)</u>, quantity, unit_price)

Payment(payment_id(PK), order_id(FK), payment_date, payment_method, payment_amt, payment_status)

Shipment(shipment_id(PK), order_id(FK), shipped_date, delivered_date, tracking_number, carrier)

Inventory(inventory_id(PK), product_id(FK), warehouse_location, quantity_on_hand)

Review(review_id(PK), customer_id(FK), product_id(FK), rating, review_text, created_at)

3NF/ Denormalization

All relations in my schema are in 3NF.
Each table contains attributes that depend only on the primary key, and there are no transitive dependencies.

The only intentional denormalization is in the CUSTOMER_ORDER table, where I store a total_amount attribute. This value could be recomputed from the related ORDER_ITEM rows, but keeping it directly in CUSTOMER_ORDER improves performance for frequent reporting queries (monthly revenue, dashboards). Because total_amount is updated whenever order items change, this denormalization is acceptable and does not introduce data inconsistency issues when handled at the application/DB layer.

All other tables remain fully normalized for data integrity and to avoid redundancy.

Relation with the most UPDATE operations

CUSTOMER_ORDER is the table that will experience the most UPDATE operations.
Order records change multiple times throughout the purchase lifecycle. For example, the order_status field is updated as an order moves from PLACED → PAID → SHIPPED → DELIVERED, and may also change to CANCELLED or RETURNED. In addition, total_amount may be updated if items are added, removed, or partially refunded before payment is finalized. Because order processing is dynamic, CUSTOMER_ORDER naturally receives the highest volume of updates.

Relation with the fewest UPDATE operations

The CATEGORY table will have the fewest updates.
Product categories (e.g., Tops, Shoes, Accessories) are created infrequently and rarely modified once defined. Most operations on this table are SELECT queries used for filtering or reporting. Compared to transactional tables, CATEGORY is highly stable and experiences minimal changes over time.

Table Where ACID Properties Are Most Important

ACID guarantees are most critical for the PAYMENT table.
Each row represents a financial transaction, which must be stored consistently and accurately to avoid double charges, missing charges, or incorrect account balances. Atomicity ensures that payments are either fully recorded or not recorded at all. Consistency maintains proper linkage between payments and customer orders. Isolation prevents concurrent payment operations from interfering with each other. Durability guarantees that once a payment is committed, it cannot be

lost. Because inaccurate payment records have real financial consequences, strict ACID adherence is essential.

Two Data Integrity Constraints

1. Product Price and Inventory Cannot Be Negative
   - **PRODUCT.price > 0**
   - **INVENTORY.quantity_on_hand >= 0**

   These constraints prevent logically invalid values and ensure financial and operational accuracy.

2. Customer Review Ratings Must Be in a Valid Range
   - **REVIEW.rating** must be an integer between 1 and 5

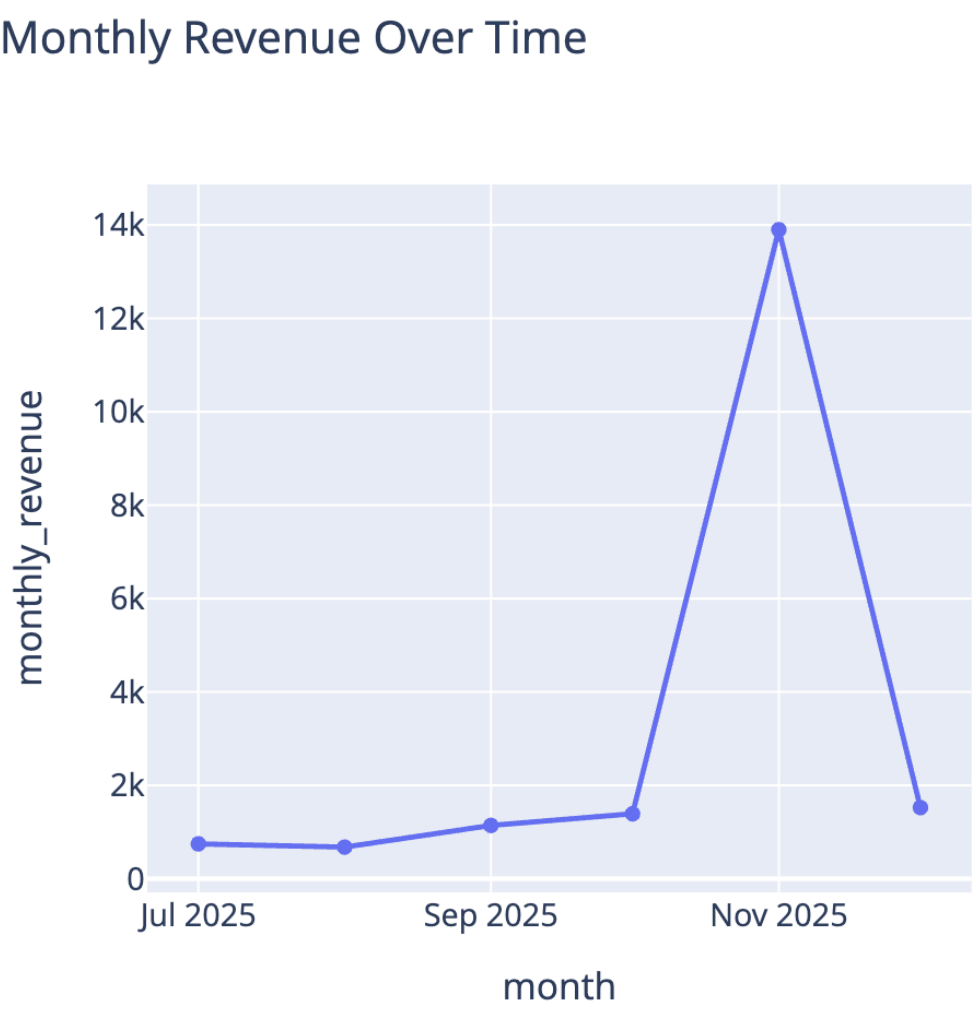
   This enforces a consistent and interpretable rating system across all products.

SQL Queries on my data

1. Monthly Revenue Over Time

This SQL query calculates total revenue for each month by grouping orders based on their order date. The resulting chart shows a large spike in November 2025 because most of the synthetic orders in the dataset were dated in November. This creates a realistic seasonal pattern resembling Black Friday or holiday shopping peaks.

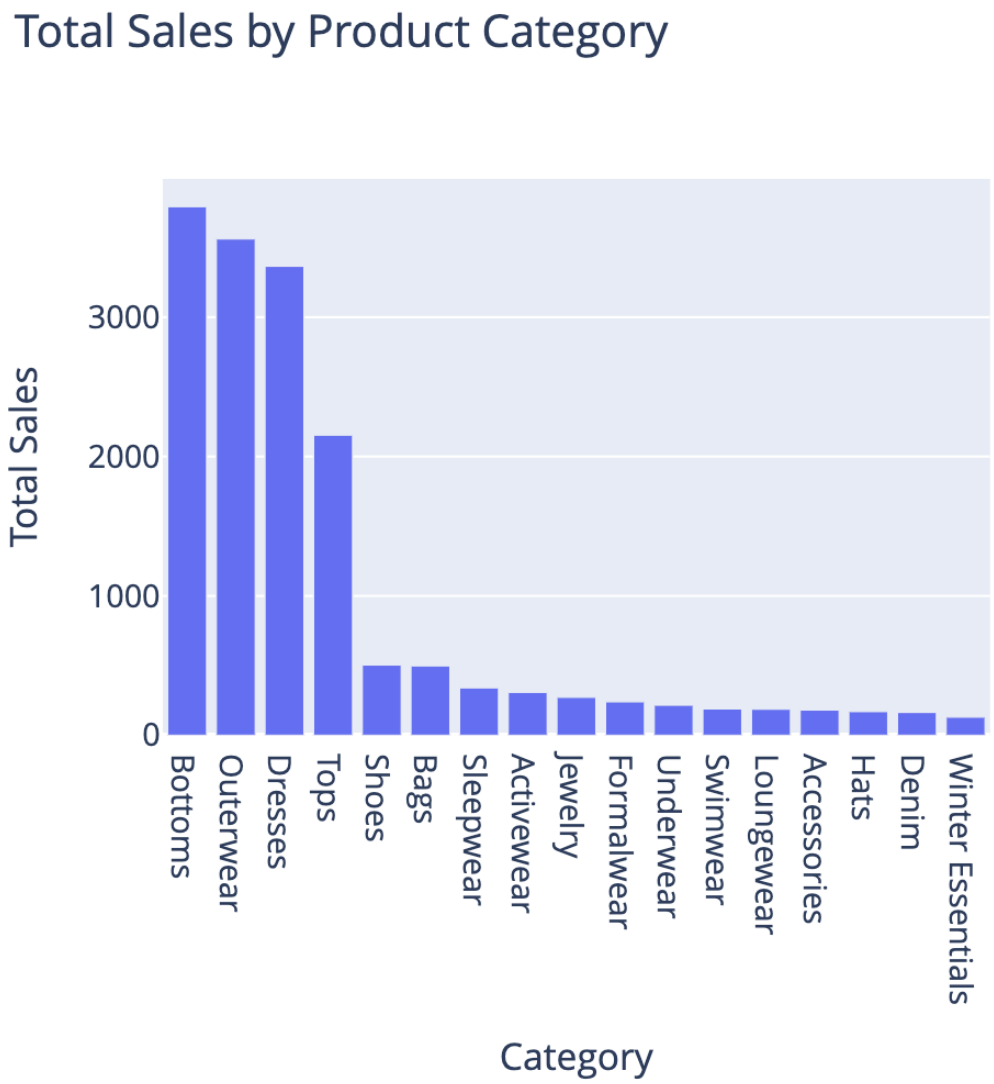
```
SELECT
  DATE_TRUNC('month', order_date) AS month,
  SUM(total_amount) AS monthly_revenue
FROM customer_order
GROUP BY month
ORDER BY month;
```



2. Top Selling Product Categories

This SQL query calculates total revenue for each product category. It joins the order items, products, and categories tables so each line item can be linked to its category, multiplies quantity by unit price to compute sales per item, and then sums those values for each category. The resulting bar chart highlights which categories generate the most revenue, with Bottoms, Outerwear, and Dresses emerging as the top performers in the dataset.

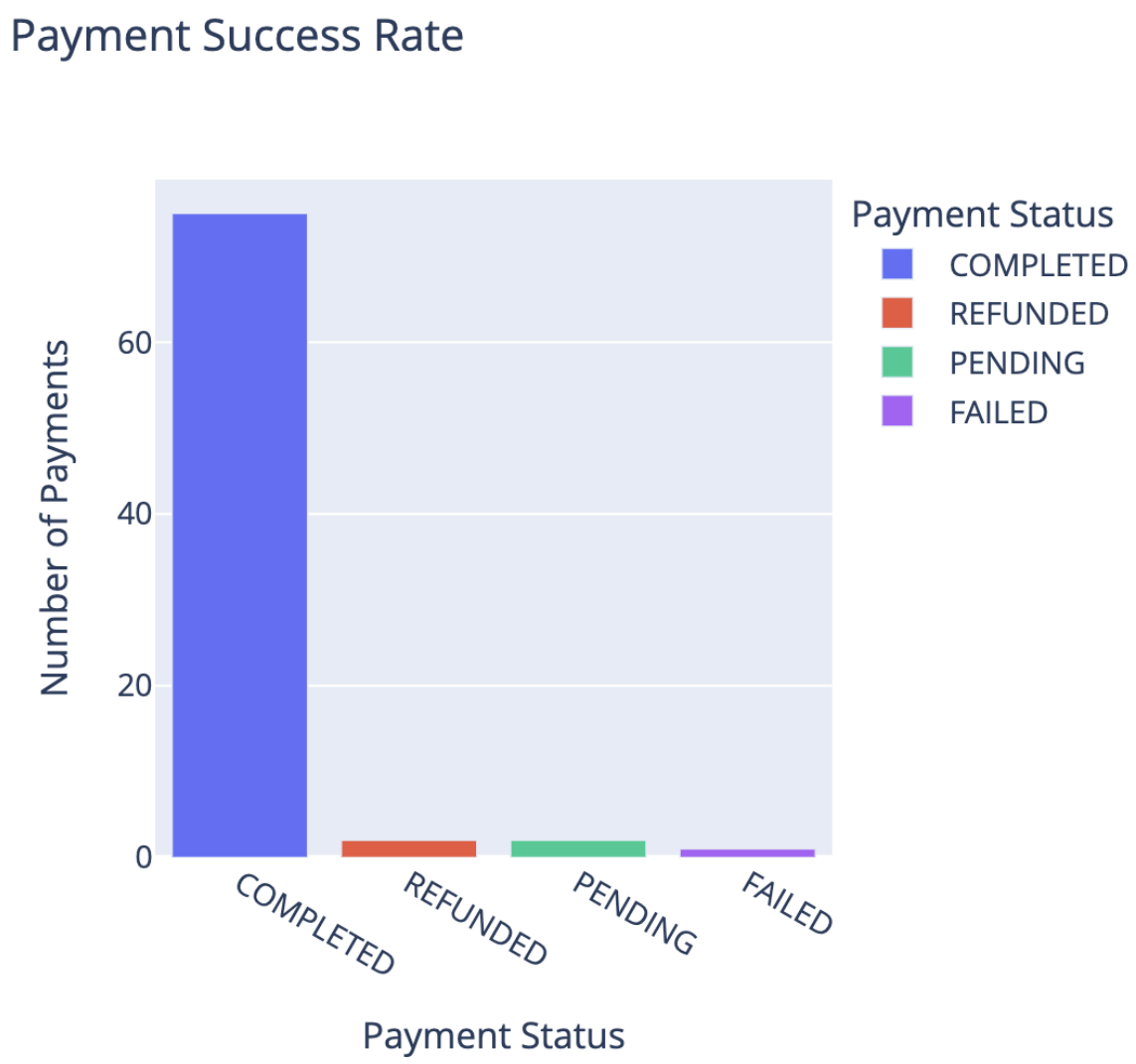
```
SELECT
  c.category_name,
  SUM(oi.quantity * oi.unit_price) AS total_category_sales
FROM order_item oi
JOIN product p
  ON oi.product_id = p.product_id
JOIN category c
  ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY total_category_sales DESC;
```



3. Payment Success Rate

This SQL query summarizes how many payments fall under each status (Completed, Pending, Failed, and Refunded). It groups all transactions by their payment_status field and counts the number of records in each category. The resulting bar chart shows the distribution of payment outcomes, making it easy to evaluate overall payment success rates.

```
SELECT
  payment_status,
  COUNT(*) AS count_status
FROM Payment
GROUP BY payment_status
ORDER BY count_status DESC;
```



All the SQL CREATE TABLE statements

```
-- 1. CUSTOMER
CREATE TABLE Customer (
  customer_id    SERIAL PRIMARY KEY,
  first_name     VARCHAR(50) NOT NULL,
  last_name      VARCHAR(50) NOT NULL,
  email          VARCHAR(255) UNIQUE NOT NULL,
  phone          VARCHAR(30)
);
-- 2. CATEGORY
CREATE TABLE Category (
  category_id    SERIAL PRIMARY KEY,
  category_name  VARCHAR(100) UNIQUE NOT NULL,
  description    TEXT
);
-- 3. PRODUCT
CREATE TABLE Product (
```

```sql
  product_id   SERIAL PRIMARY KEY,
  category_id  INTEGER NOT NULL REFERENCES Category(category_id),
  product_name VARCHAR(150) NOT NULL,
  price        NUMERIC(10,2) NOT NULL,
  CONSTRAINT chk_product_price_positive
     CHECK (price > 0)   -- CHECK #1
);
-- 4. CUSTOMER_ORDER
CREATE TABLE Customer_Order (
  order_id      SERIAL PRIMARY KEY,
  customer_id   INTEGER NOT NULL REFERENCES Customer(customer_id),
  order_date    TIMESTAMP NOT NULL,
  order_status  VARCHAR(20) NOT NULL,
  total_amount  NUMERIC(12,2) NOT NULL,
  CONSTRAINT chk_order_status_valid
     CHECK (order_status IN (
        'PLACED', 'PAID', 'SHIPPED', 'DELIVERED', 'CANCELLED', 'RETURNED'
     ))
);
-- 5. ORDER_ITEM
CREATE TABLE Order_Item (
  order_item_id SERIAL PRIMARY KEY,
  order_id      INTEGER NOT NULL REFERENCES Customer_Order(order_id),
  product_id    INTEGER NOT NULL REFERENCES Product(product_id),
  quantity      INTEGER NOT NULL,
  unit_price    NUMERIC(10,2) NOT NULL
);
-- 6. PAYMENT
CREATE TABLE Payment (
  payment_id     SERIAL PRIMARY KEY,
  order_id       INTEGER NOT NULL REFERENCES Customer_Order(order_id),
  payment_date   TIMESTAMP NOT NULL,
  payment_method VARCHAR(30) NOT NULL,
  payment_amt    NUMERIC(12,2) NOT NULL,
  payment_status VARCHAR(20) NOT NULL

);
-- 7. SHIPMENT
CREATE TABLE Shipment (
  shipment_id     SERIAL PRIMARY KEY,
  order_id        INTEGER NOT NULL REFERENCES Customer_Order(order_id),
  shipped_date    TIMESTAMP,
  delivered_date  TIMESTAMP,
  tracking_number VARCHAR(100) UNIQUE,
  carrier         VARCHAR(50)
);
-- 8. INVENTORY
CREATE TABLE Inventory (
```

```sql
  inventory_id      SERIAL PRIMARY KEY,
  product_id        INTEGER NOT NULL REFERENCES Product(product_id),
  warehouse_location VARCHAR(100) NOT NULL,
  quantity_on_hand  INTEGER NOT NULL,
  CONSTRAINT chk_inventory_nonnegative
     CHECK (quantity_on_hand >= 0)   -- CHECK #2
);
-- 9. REVIEW
CREATE TABLE Review (
  review_id    SERIAL PRIMARY KEY,
  customer_id  INTEGER NOT NULL REFERENCES Customer(customer_id),
  product_id   INTEGER NOT NULL REFERENCES Product(product_id),
  rating       INTEGER NOT NULL,
  review_text  TEXT,
  created_at   TIMESTAMP NOT NULL,
  CONSTRAINT chk_review_rating_range
     CHECK (rating BETWEEN 1 AND 5)   -- CHECK #3
);
```

Python Code

```python
#%%
import psycopg2
import pandas as pd
import plotly.express as px

conn = psycopg2.connect(
   host="34.94.6.101",
   database="postgres",
   user="guest",
   password=[redacted]!"
)

conn.autocommit = True
cur = conn.cursor()

table_to_drop = "customer_order"

try:
   cur.execute(f"DROP TABLE {table_to_drop} CASCADE;")
   print("Unexpected: guest was able to drop the table (should not happen).")
except Exception as e:
   print("Permission test successful: guest cannot drop tables.")
   print("Database error message:", e)

# Verify that the table still exists
cur.execute("""
```

```
   SELECT EXISTS (
       SELECT 1
       FROM information_schema.tables
       WHERE table_schema = 'public'
         AND table_name = %s
   );
""", (table_to_drop,))
exists = cur.fetchone()[0]

if exists:
   print(f"Confirmed: table '{table_to_drop}' still exists.")
else:
   print(f"Table '{table_to_drop}' does NOT exist — something is wrong.")



#%%
# Monthly Revenue Over Time
import pandas as pd
import plotly.express as px

query = """
SELECT
   DATE_TRUNC('month', order_date) AS month,
   SUM(total_amount) AS monthly_revenue
FROM customer_order
GROUP BY month
ORDER BY month;
"""

df = pd.read_sql_query(query, conn)

print("\nMonthly Revenue:")
print(df)

fig = px.line(
   df,
   x="month",
   y="monthly_revenue",
   title="Monthly Revenue Over Time",
   markers=True
)
fig.show()

#%%
# Top Selling Product Categories

import pandas as pd
import plotly.express as px
```

```
query2 = """
SELECT
   c.category_name,
   SUM(oi.quantity * oi.unit_price) AS total_category_sales
FROM order_item oi
JOIN product p
   ON oi.product_id = p.product_id
JOIN category c
   ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY total_category_sales DESC;

"""

df2 = pd.read_sql_query(query2, conn)

print("\nTop-Selling Product Categories:")
print(df2)

fig2 = px.bar(
   df2,
   x="category_name",
   y="total_category_sales",
   title="Total Sales by Product Category",
   labels={"category_name": "Category", "total_category_sales": "Total Sales"},
)

fig2.show()



# %%
# Payment Success Rate
import pandas as pd
import plotly.express as px

query3 = """
SELECT
   payment_status,
   COUNT(*) AS count_status
FROM Payment
GROUP BY payment_status
ORDER BY count_status DESC;
"""

df3 = pd.read_sql_query(query3, conn)

print("\nPayment Status Counts:")
print(df3)
```

```
fig = px.bar(
   df3,
   x="payment_status",
   y="count_status",
   title="Payment Success Rate",
   labels={"payment_status": "Payment Status", "count_status": "Number of Payments"},
   color="payment_status",
)
fig.show()

# %%
```